VROSTBridge

Full project specification

An open-source remote PCVR bridge
VR Rendering OutSide The Box – Bridge
Windows rendering host **1.0.0.15**
Bazzite headset client **1.0.0.15**
Windows headset client **1.0.0.15**
MIT licence, by JagerBombastic

Current handoff
Architecture, requirements, implementation boundaries, build and install contracts, and release acceptance for 1.0.0.15: the current design and the reasons for it. Recent changes are in the release notes

Validation boundary
Index and Beyond 2e streaming, tracking and game audio are confirmed on hardware through the bridge, with eye-tracked foveation at the headset's full resolution, calibration over the stream in Desktop Mode, packet repair and the menu button. The other features are contract-tested and listed as pending in section 16.2. The Windows client is built and tested in code only and has not yet run on Windows hardware; Game Mode sessions, calibrated gaze in Game Mode and the Beyond projection check are the remaining gates.

Contents

1. Purpose and scope 3

2. Release baseline and evidence 5

3. Product requirements 6

4. System architecture 7

5. Session lifecycle and ownership 10

6. Optical geometry and display negotiation 11

7. Video quality and frame delivery 12

8. Tracking, input, menus and play area 17

9. Audio and optional eye tracking 17

10. Network and trust model 19

11. User interfaces and Game Mode 20

12. Installation, upgrades and stored state 21

13. Diagnostics and support 21

14. Build, dependencies and source handoff 22

15. Attribution and publication readiness 23

16. Validation and release acceptance 24

17. Priorities and future work 26

Canonical text: docs/PROJECT_SPEC.md in the source archive. The PDF presents that specification with a cover and navigation; dated release reports remain in docs/evidence/.

Documentation revision: **2026-11-13 / 1.0.0.15**. Runtime baseline: **VROSTBridge Windows host, Bazzite client and Windows client 1.0.0.15**, one version for every package; the host's driver is the 1.0.0.15 build (ALVR patches 0001-0023). Licence: MIT (VROSTBridge's own code; `LICENSE`, `NOTICE`, `THIRD_PARTY.md`). Status: first release line. Hardware support is limited to the combinations in HARDWARE_ACCEPTANCE.md; features marked pending below are implemented and contract-tested but not yet accepted on hardware, and the Windows client has not yet run on Windows hardware. What changed recently, and why, is in RELEASE_1.0.0.15.md, RELEASE_1.0.0.14.md, RELEASE_1.0.0.13.md, RELEASE_1.0.0.12.md and RELEASE_1.0.0.11.md; older notes are in `history/`.

This document states the current design and the reasons for it. It describes what the software does, not every step of every test; release notes and `docs/evidence/` carry the test detail.

1. Purpose and scope

VROSTBridge lets a powerful Windows PC render SteamVR applications for a DisplayPort headset attached to a separate headset PC, the client, on the same trusted home network. The client is a Bazzite Linux PC or a Windows 10 or 11 PC. It handles the headset's display, Lighthouse tracking, controller input, video decoding and audio playback. The rendering PC, the host, runs the game, virtual SteamVR devices and video encoding. The aim is a convenient, low-latency VR session started from the headset PC, with stable image quality and correct headset geometry.

The supported product scope is Valve Index and Bigscreen Beyond-family headsets, including the Beyond 2e's eye tracking, with Index controllers and Lighthouse base stations. The headset's own SteamVR driver on the client remains responsible for device support; a family match does not prove every model, refresh rate or adapter has been qualified.

The design is general rather than tied to one PC, dock or network. Any x86-64 headset PC whose GPU can decode the stream and drive the headset through SteamVR can be a client; handheld PCs with integrated graphics are the demanding case the design is tuned for (power limits, shared memory bandwidth, Wi-Fi). The rendering PC, client GPU, dock, cable, refresh rate and network must each support the chosen workload. Ethernet and Wi-Fi are both supported. A working local headset setup is assumed before remote streaming is diagnosed.

On hardware, Index and Beyond 2e streaming, tracking and audio have been confirmed through the bridge, as have eye-tracked foveation at the headset's full resolution, packet repair, clean game audio and the menu button. This specification distinguishes implemented software from hardware acceptance throughout.

1.1 Boundaries

- Windows is the rendering host. The headset client runs on Bazzite x86-64 or Windows 10/11 x64: one source, two builds (section 4.4). Game Mode, the headset display lease and the eye-camera helper are Bazzite's.
- The Windows client is built and contract-tested only; it has not run on Windows hardware.
- The product targets a trusted private LAN, not Internet streaming, hostile networks or multi-tenant hosting.
- One client owns one active host session. Multi-headset rendering is outside scope.
- Steam, SteamVR, headset drivers and graphics drivers are separate prerequisites. Gaze inference (Baballonia) is a separate program under its own licence that VROSTBridge downloads, configures and runs only when asked.
- In-session changes of stream size or codec, HDR, host OpenXR gaze support and dynamic foveated rendering (games rendering fewer pixels) are not implemented. Foveated encoding, fixed or driven by Beyond 2e gaze, is; the bitrate follows the stream during a session.

1.2 Name

The project is **VROSTBridge** (An open-source remote PCVR bridge; programs **VROSTBridge Host** and **VROSTBridge Client**); the name stands for VR Rendering OutSide The Box - Bridge. Its logo is in `brand/`. Every identifier carries the name: the state and install folders (`%LOCALAPPDATA%\VROSTBridge`, `~/.local/share/vrostbridge`, `~/.local/state/VROSTBridge`), Windows uninstall keys and Run values (`VROSTBridgeHost`, `VROSTBridgeClient`), firewall rules (`VROSTBridge.*`, `VROSTBridge.Client.*`) and QoS policy (`VROSTBridgeVideo`), the programs (`VROSTBridge-Client`, `VROSTBridgeClient`, `VROSTBridgeLease`, `VROSTBridgeRuntimeProbe`, `VROSTBridge Client.exe`), the transport library (`libvrostbridge_transport.so`, `vrostbridge_transport.dll`), `libvrosttext.so`, desktop entries (`io.vrostbridge.*`), the session and native protocol names (`vrostbridge-session-2`, `vrostbridge-native-1`) and headers (`X-VROSTBridge-*`), the driver's files (`vrostbridge-stream.json` and the others) and log lines (`VROST_*`), the emulated headset serial, environment variables (`VROSTBRIDGE_*`) and the code (`vrost::`). `tests/test_vrostbridge_1001.py` checks that the names shared by the companion, the client, the transport and the driver agree.

2. Release baseline and evidence

Area	Current state	Evidence
Windows host	Python companion (tray, page, session broker, network test sender, reports, keep-awake, Wake-on-LAN readiness, sleep after session, quick resume, optional QoS policy, VB-CABLE microphone setup, SteamVR's safe-mode check and its fix). Driver built from ALVR plus patches 0001-0023 without ALVR's `gpl` feature: frame-timing lock, pacing, gaze and optical-centre foveated encoding with prefilter, hold and comfort zone, AMF intra refresh and output drain, prediction switch, startup display configuration, exact colour with its plain-SDR variant (`0020-host-colour-variants-and-gaze-reach.patch`), parity for video and audio, the game-audio capture with the device's volume applied. The game is rendered at the headset PC's recommended size (at most 1.5 × the stream) and scaled down for the encoder; the last session's statistics are kept for the report	Index streaming, Beyond 2e streaming, game audio, packet repair and live bitrate changes confirmed on hardware; driver cross-built and checked offline (exports, imports, manifest), its VROSTBridge Rust modules and colour rules tested on Linux
Bazzite client	Native VA-API decode, one-pass Vulkan frame pass (re-aim, foveation inverse, sharpening, colour, lens-area skip, smaller shown size under load), single-step prediction with controller acceleration and gaze prediction, packet repair, own game-audio player, in-headset cards; companion with network test, bitrate governor, performance balance through steamos-manager, network settings for a PC on its network twice, base stations, managed eye tracking, Game Mode integration, reports and timeline	Native, Python, UI and package checks, software-Vulkan runs of the frame path; Desktop Mode sessions on hardware with Index and Beyond 2e, eye-tracked foveation at Native, calibration over the stream; Game Mode handover pending
Windows client	The same client built for Windows: D3D11VA decoding on SteamVR's GPU copied into the same frame pass, tray icon and sign-in start, firewall rules, Wi-Fi and power handled for the session, base stations, Baballonia's Windows release, per-user installer	Built and contract-tested only: Python tests with stand-ins for Windows calls, native contract tests under Wine with a software Vulkan driver, installer and companion smoke runs under Wine; not yet run on Windows hardware
Valve Index	Video, tracking, audio and input through the bridge; Native at 80/90/120/144 Hz chosen before a session	Confirmed at its default rate; 120 and 144 Hz and a long controller soak pending
Beyond family	Streams the mode it is in; projection from raw tangents; 90/75 Hz refresh exception	Streaming at 75 Hz confirmed; projection visual check pending
Game Mode (Bazzite)	Native SDL interface with every setting, Steam entry, headset display reserved by EDID and leased to SteamVR	Interface confirmed; SteamVR's compositor taking the leased display pending
Beyond 2e gaze	One switch: Baballonia downloaded (pinned), configured, run and recovered with the camera helper and relay; calibration over the stream; gaze-driven foveation	Calibration over the stream and eye-tracked foveation confirmed in Desktop Mode; Game Mode and Windows paths pending

Release evidence is kept under `docs/evidence/`: per-release build records (binary hashes, reused dependencies, test results) and the package records for managed upgrade, repair, retained state, launcher self-test and report access. They are not substitutes for physical acceptance.

3. Product requirements

ID	Requirement	Status
REQ-01	Start and stop both ends from the client	Implemented; hardware-confirmed
REQ-02	Keep the host idle in the tray, with Settings, Exit and status (the Windows client has its own tray)	Implemented; hardware-confirmed (host); Windows client pending
REQ-03	Optional sign-in startup opens neither a browser nor SteamVR	Implemented; hardware-confirmed (host); Windows client pending
REQ-04	Discover and pair automatically on a trusted LAN	Implemented; hardware-confirmed
REQ-05	Detect the headset, stream dimensions, refresh and eye geometry	Implemented; hardware-confirmed; Beyond projection visual check pending
REQ-06	Forward both Index controllers, skeletal input, menus and haptics	Implemented; hardware-confirmed; long soak and reconnect matrix pending
REQ-07	Keep the local compositor and tracking active without a competing local Home scene	Implemented; hardware-confirmed
REQ-08	Desktop web UI and native Bazzite Game Mode controls; never use a headset as the Game Mode screen	Implemented; desktop hardware-confirmed; Game Mode session pending
REQ-09	Transfer supported play-area geometry to the host	Implemented (rectangle and standing transform); pending
REQ-10	Preserve preferences and remove only owned obsolete files during upgrades	Implemented; hardware-confirmed
REQ-11	Export actionable reports, including client diagnostics kept on the host	Implemented; hardware-confirmed
REQ-12	Preserve image quality while reducing latency and uneven frame delivery	Implemented; hardware-confirmed, tuning continues
REQ-13	Allow IPD and eye-geometry updates during a session	Implemented; pending
REQ-14	Opt-in Beyond 2e calibrated gaze, set up and run by VROSTBridge	Implemented; hardware-confirmed in Desktop Mode; Game Mode and Windows pending
REQ-15	Preserve upstream attribution and meet distribution obligations	Implemented (section 15)
REQ-16	Spend the stream budget where the user looks (foveated encoding)	Implemented; hardware-confirmed
REQ-17	Keep the host usable from another room: awake in sessions, wakeable, may sleep after use	Implemented; pending
REQ-18	Choose a bitrate and pacing the measured network path carries, and follow it during a session	Implemented; hardware-confirmed

ID	Requirement	Status
REQ-19	Tell the user in the headset what is happening when video stops	Implemented; hardware-confirmed
REQ-20	Bring the stream back after a network interruption without user action	Implemented; pending
REQ-21	Keep head-locked content steady while the head moves	Implemented; hardware-confirmed
REQ-22	Keep Wi-Fi from pausing the stream to scan, and keep a wired stream on the cable	Implemented; pending
REQ-23	Make the headset microphone available to Windows apps	Implemented; pending
REQ-24	Tell the user when the headset PC cannot present at the headset refresh, and react	Implemented; partly hardware-confirmed (smaller shown size pending)
REQ-25	Run the headset client on Windows 10/11 x64 with the same page, session flow, stream and eye tracking	Implemented; pending (built and contract-tested only)
REQ-26	One version for every package, installed together	Implemented (`tests/test_vrostbridge_1007.py`)

4. System architecture

4.1 Windows rendering host

SteamVR runs a patched ALVR virtual headset and controller driver. Games render for that virtual headset. ALVR captures and hardware-encodes the view (optionally foveated: edges compressed before encoding, with the centres sent per frame) and sends video and audio to the native client. Tracking, controller and optional gaze data flow back. The virtual device's properties follow the negotiated physical headset where supported, including native Index identity and controller mappings. This improves SteamVR's behaviour but does not make the virtual device a USB-connected headset, so vendor firmware tools and every dashboard option are not guaranteed to work remotely.

The Python companion owns the host profile, driver registration, lifecycle, diagnostics and desktop UI. Per-session driver options that must not change ALVR's session schema (pacing, intra refresh, foveation, parity, the timing lock) are written to `vrostbridge-stream.json` and `vrostbridge-timing.json` beside `session.json`, which the driver reads at each client connection; the companion writes each option only for a driver whose build manifest lists the patch, so mixed versions keep working. A native Windows launcher starts the bundled Python runtime; the tray uses Win32. Helpers apply chaperone geometry through OpenVR and check the microphone endpoint. No user-installed Python is needed.

4.2 Linux headset client

The local SteamVR owns the headset, Lighthouse devices, local tracking, chaperone and final lens/display presentation. VROSTBridge's C++ client receives frames, decodes them with FFmpeg (VA-API; NV12 or 10-bit P010), converts and resamples them in one Vulkan compute pass, and submits per-eye textures with their source pose to the local compositor. A head-locked OpenVR overlay shows status cards when video is not fresh. A Rust C-ABI adapter wraps the pinned ALVR client core for streaming, tracking and audio.

Keeping SteamVR on the client is deliberate: removing its compositor and driver stack would need a different local runtime integration. The client stays light by suppressing SteamVR Home and the local dashboard, bounding work and queues, and decoding in hardware; it does not claim zero SteamVR overhead.

The shared Python companion supplies the desktop page and the Game Mode interface, starts the runtime, negotiates the headset profile, stores preferences and collects diagnostics. `VROSTBridgeRuntimeProbe` verifies the physical runtime before a host stream is requested. `VROSTBridgeLease` bridges the gamescope display lease to SteamVR in Game Mode.

4.3 Source ownership

Location	Responsibility
`client/platform/linux/`	Native receive/decode/present loop, Vulkan, shaders, runtime probe, DRM-lease helper, the OpenVR texture layer; also builds the Windows client, with Windows sections
`client/platform/windows/`	D3D11VA decoding (`d3d11_video.*`), upload into Vulkan, command pipe and process plumbing, version resource and manifest
`client/device/`	OpenVR tracking and pose schedule, acceleration and gaze prediction, optical geometry, visible area, input, readiness, plane upload, gaze input
`client/ui/`	Text and shape rasteriser, in-headset cards, `libvrosttext.so` for Game Mode
`client/transport/`	Pinned ALVR client-core adapter and C ABI
`core/`	Shared frame, geometry, presentation and input contracts
`host/`	Host profile, lifecycle, audio, telemetry and native helpers
`launcher/`	Page and Game Mode UI, tray, startup, session broker, network test, governor, performance and network settings, eye tracking, reports and maintenance, Windows branches
`patches/`	Twenty-three reviewed modifications to the pinned ALVR (section 14)
`packaging/`, `scripts/`	Installers, builds (including cross-builds and `scripts/cross/` toolchains), provenance and source archives
`tests/`, `docs/evidence/`	Automated checks and retained validation records

4.4 Windows headset client

The Windows client is the same client built for Windows 10 or 11 (x64). SteamVR still owns the headset and the native client is its scene application: the same ALVR client core behind the same C interface (no further ALVR patch), the same frame pass, foveation inverse, sharpening, colour and in-headset cards. The companion is the same Python program in the client role, started by `VROSTBridge Client.exe` on an embedded CPython 3.13 taken byte for byte from the host's package. It serves the same page; with `platform: windows` in its status the page uses Windows wording and hides Bazzite's controls (Game Mode, Add to Steam, the display lease, eye-camera access, Fix SteamVR reprojection, the GPU clock fix), whose actions, if called, say why they do not apply.

Decoding. FFmpeg n6.1.4 as Windows DLLs (H.264, HEVC and AV1 with D3D11VA, LGPL, cross-compiled with mingw-w64 from the same unmodified source as Bazzite's). The decoder runs on the adapter whose LUID SteamVR's Vulkan GPU reports, checked again once FFmpeg has opened it. Its

D3D11 decoder profiles decide what the client offers: 10-bit only with an HEVC Main10 decoder that outputs P010, AV1 only with an AV1 decoder, and Auto codec HEVC wherever HEVC is decoded. Each picture is downloaded into a mapped staging buffer and copied into the same plane images the Bazzite client fills; the copy time is reported as `map_sync_import`. CPU decoding exists for troubleshooting.

Process and gaze. The companion starts the native client with its standard input as a command pipe and stops it by writing `quit`, then closing the pipe after 8 s. Gaze arrives on a UDP port on 127.0.0.1 chosen at each Start. SteamVR is found through OpenVR's registry (`openvrpaths.vrpath`), started with `vrstartup.exe` and asked to quit with `vrmonitor://quit`; a PC where SteamVR never ran gets a plain message.

Windows' own settings, for the session. A tray icon (Open VROSTBridge, Start session, Stop session, Quit) with notifications for a session started there; Start when I sign in (Run value `VROSTBridgeClient`, tray only); five inbound firewall rules for exactly the Bazzite client's ports, each for the program that listens, any network profile, local subnet only, after one UAC prompt; Wi-Fi in media streaming mode with background scanning off, other Wi-Fi adapters on the same network paused for a Wi-Fi stream and left connected for a wired one (Windows answers ARP only on the adapter that has the address); Best performance power mode with a receipt; crash records from Windows' event logs in reports. SteamVR switches audio to the headset itself, so the client changes no audio device. Windows 11 24H2 and later name the Wi-Fi network only to apps with location permission; without it roaming detection and the Wi-Fi pause are off and streaming is unaffected.

Decisions

- **One client, two builds.** The Windows client is the Linux client's source with Windows sections, so the stream, re-aim, foveation, colour and every page action behave the same and are tested once. Only decoding, process plumbing, the camera path and Windows' own settings differ.
- **A copy first, sharing later.** FFmpeg has no mapping from a Direct3D 11 picture into Vulkan, so each picture is copied through a staging buffer into the tested frame pass's planes. Sharing the decoder's texture, or Vulkan Video decoding, is future work if reports show the copy matters.
- **SteamVR's GPU, by LUID.** Decoding on another GPU would move every picture between GPUs, and FFmpeg falls back to the default adapter silently, so the LUID is checked and the offered formats follow that adapter.
- **Gaze over loopback UDP,** `quit` **on stdin.** Windows' Python has no Unix datagram sockets and Windows has no SIGTERM; ending the process would skip SteamVR's and the transport's clean-up.
- **No Game Mode on Windows.** SteamVR on Windows drives the headset itself and the companion lives in the tray, so gamescope's display reservation and lease are not needed.
- **Firewall rules no wider than Bazzite's.** The same ports, per program, local subnet only, removed by the uninstaller.
- **Baballonia private, the user's own respected.** Project Babble publishes Baballonia for Windows only as an installer; VROSTBridge runs it silently into its own folder in a job object and puts back the entries it wrote, or uses a matching installation the user already has. It sets Baballonia's eye model to the processor once, since the GPU's time belongs to decoding and the compositor.
- **No Visual C++ runtime, one Python.** The native programs link the C runtime statically; the companion's Python is the host package's, verified byte for byte.
- **Cross-built on Linux** with clang-cl and lld-link against an xwin-style SDK, with build paths mapped out and content-derived timestamps, so no Windows build machine is needed; the C++ tests run under Wine.

5. Session lifecycle and ownership

5.1 Idle and discovery

The host companion waits in the tray; Settings opens the local page and Exit closes it cleanly. Sign-in startup stays quiet and does not launch SteamVR. The client is the normal Start authority; the Windows client waits the same way in its own tray.

The client discovers a host on the selected private LAN, and stored pairing credentials permit later sessions. Only automatic pairing of a new client requires the rendering PC's network to be Private (or domain) in Windows (`pairing_network_private`); paired clients start sessions on any connected network (`require_home_network`), because Windows reclasses a home network as Public whenever it decides it is new, which used to stop every session. The firewall rules (`-Profile Any -RemoteAddress LocalSubnet`) still admit only the local subnet, and a stranger on a public network still cannot pair. The software never changes the Windows network category or disables a firewall. With several adapters, the intended Ethernet or Wi-Fi route can be chosen.

5.2 Starting

1. The user chooses stream settings and presses Start on the client's page, tray or Game Mode screen.
2. The client records ownership of temporary runtime settings, starts the local SteamVR if needed and probes a ready headset with advancing display timing. When a headset was verified before, the host lease is obtained in parallel with that cached profile; if the fresh headset differs, the host session is restarted with the detected profile before streaming.
3. Headset identity, render recommendation, effective refresh and stream preferences are validated. Auto settings resolve for this session without overwriting saved explicit choices.
4. The companion wakes a sleeping host with Wake-on-LAN when its address is known and, with automatic bitrate, runs the network test once per host and adapter. It obtains a host session lease. The host writes the client's display configuration for the driver before SteamVR starts (patch 0015), so the virtual headset is right from the start and no restart is needed. A host kept open for quick resume is reused when the settings match.
5. The native client connects, receives the video configuration and enters streaming only when usable frames arrive. Connection, decoder and presentation states stay distinct in diagnostics.

Failures are reported by stage. A running process, a headset icon or a discovered host is not proof that video flows. Cancelling releases the host lease and stops what the attempt started.

5.3 Running and stopping

The companion protocol uses a 3-second heartbeat and a 30-second host lease, so a lost companion has a bounded ownership window. Heartbeats carry the client's stream state; while it is current, the host restarts its owned SteamVR session when the stream stays down (8 s with ALVR stuck disconnecting, 20 s while ALVR still shows the client connected, 45 s otherwise; at most twice per session, three minutes apart). ALVR retries the client's known address every second. Start requests are idempotent and another client cannot stop an owner's session.

Stop ends streaming and closes the SteamVR runtimes managed for the session on both PCs; local settings are restored from ownership receipts without overwriting unrelated edits. With quick resume (off by default) the host's SteamVR stays open for a chosen 5-60 minutes. The host stays awake during sessions and may sleep a chosen time afterwards. A stop while the rendering PC's SteamVR is still starting does not wait out the start: the shutdown window counts from when SteamVR's server appears, and the host asks SteamVR to quit itself (`vrmonitor://quit`) when ALVR cannot, since ALVR can ask only through a streaming client's headset.

The host owns only the SteamVR it started: the runtime family's processes from that installation created after its launch, identified by PID, image path and creation time. A new `vrserver` of that installation joins the session while it runs; any other new server ends the session and is never stopped.

Compositor crash recovery. If SteamVR's compositor on the client crashes, nothing can be shown in the headset. The native client ignores SIGPIPE, reports frames that stop advancing, and exits when the companion's command pipe closes. The companion watches this session's exact SteamVR processes once video is fresh; a lost compositor or server, confirmed over two checks, closes the session on both PCs and starts it again once, with restarting/restarted on the pages and Game Mode. A Stop or Start by hand always wins, and a second crash within ten minutes stops the session with the explanation. Reports collect the crash records (`coredumpctl` and kernel GPU lines on Bazzite; Application Error/Hang and display-reset events on Windows). VROSTBridge cannot prevent the crash itself.

SteamVR's safe mode on the host. SteamVR keeps add-on drivers off after a safe-mode start (`blocked_by_safe_mode` in the driver's section of `steamvr.vrsettings`, or `enable` false from its add-on manager). The host reads that before launching SteamVR and refuses with the reason, and ends a launch at once when SteamVR's server log says it skipped the driver. It turns the driver back on only on request (its page, or a paired client's `/unblock` between sessions) and only while SteamVR is closed, rewriting only that section.

The headset's hardware. The client names a USB hub Linux refuses for its depth (from sysfs, at Start), reports the headset and a controller disconnecting together as a USB dropout (instead of SteamVR's following frame error), and notes controllers connected but untracked while the headset tracks. None of this changes what a session does.

Do not run another ALVR instance alongside VROSTBridge; the software avoids unowned driver or profile changes and may ask the user to close a conflicting runtime first.

6. Optical geometry and display negotiation

6.1 Geometry contract

Use the physical runtime's per-eye raw projection tangents and complete eye-to-head transforms. Never derive a projection from a nominal panel resolution, a generic symmetric FOV, an Index template or an IPD alone. Stream resolution sets the render and encode budget, not the optical model.

The client maps OpenVR raw values to ALVR angles as left = atan(rawLeft), right = atan(rawRight), up = atan(rawBottom), down = atan(rawTop), following OpenVR's vertical naming; the host reconstructs the raw values from those angles. A sign-flipped conversion would mirror an asymmetric vertical optical centre, which symmetric tests alone do not expose, so the tests use asymmetric optics.

Full eye position and orientation are preserved. Geometry updates, including IPD changes while running, never pair a frame rendered with old optics with new eye geometry; a bounded geometry history supports source-pose recovery and invalid updates are rejected. Reports keep static `view_geometry` (raw tangents, transmitted angles, eye transforms, revision), never live tracking. The local runtime remains responsible for lens distortion and scan-out.

6.2 Refresh handling

Readiness requires valid geometry and advancing physical timing. A narrow Beyond exception accepts a reported 90 Hz with a measured 75 Hz only after at least 1.5 s and 110 progressing frames within about 3 %, since that runtime has reported the wrong rate for its 75 Hz mode; any timing, counter or mode change resets the observation. Index start-up guards stay strict.

The effective refresh propagates into the headset metadata, host negotiation and frame budgets. The software never edits the user's refresh preference to hide a mismatch. It sets SteamVR's preferred refresh rate only for a rate the user chose for an Index, for that session, and puts it back afterwards

(section 7.1). A Beyond's mode, brightness and firmware stay with the vendor's tools.

7. Video quality and frame delivery

7.1 User controls

New preferences default to Auto codec, automatic headset resolution, Auto foveated encoding, 10-bit colour and automatic bitrate (80 Mbit/s until a network test has run). Bitrate accepts 10-300 Mbit/s. Codec, bitrate, stream size and stream options are remembered per headset family; existing saved settings are kept.

Codec. Auto chooses HEVC on every headset, 10-bit where the decoder allows, and H.264 only on a headset PC whose decoder profiles report no HEVC (`launcher/headset.py automatic_codec`). The Bazzite client reads the GPU's VA-API profiles at start and the Windows client its D3D11 profiles; the companion remembers them, so the choice is right before the first Start. HEVC gives clearly more detail per bit than H.264 at the same rate, which matters most for fine texture. AV1 is an explicit choice that needs AV1 encoding on the host and decoding on the client.

Display modes. The stream follows the headset's display mode, because a stream larger than the mode shows only costs decoding. An Index offers Native (2016 × 2240 per eye, SteamVR's size for its lenses) at 80, 90, 120 or 144 Hz (144 experimental); the chosen rate is written to SteamVR's `preferredRefreshRate` for the session and put back afterwards. A Beyond offers no choice: it streams the mode it is in, 2560 per eye at 75 Hz or 1920 at 90 Hz, and the mode is switched with Bigscreen's Beyond Utility on a Windows PC. The choices are offered before a session, from the last verified headset or a known headset display, and checked again when SteamVR describes the headset. Legacy setting identifiers stay stable, so older saved choices and older hosts keep working.

Choice / scope	Per-eye width	Constraint
Index Native at 80, 90, 120, 144 Hz	2016	H.264 or HEVC; the rate set for the session
Beyond in its 75 Hz mode	2560, the panels' own	HEVC or AV1; 1920 with H.264
Beyond in its 90 Hz mode	1920	Any codec
Other headsets: Standard / Custom	1440 with H.264; 2144 with HEVC	Auto
Other headsets: High	2016	H.264 or HEVC
Other headsets: SteamVR app size	The local runtime's recommendation	Capped and aligned for the codec

Widths and heights are aligned to 32 pixels; height follows the runtime's aspect ratio. The H.264 cap is 2048 per eye and HEVC's 4096; these are software bounds, not a promise that every GPU can encode or decode them at the chosen refresh. The negotiated and decoded dimensions are authoritative and shown apart from the requested ones.

Render size. The host renders the game at the headset PC's recommended width, at most 1.5 × the stream width and aligned to 32 (`host/profile.py render_width`), and the encoder scales it down to the stream size, which stays the transcoding size. A headset on its own PC renders larger than its panels to make up for lens distortion; rendering at the stream size had already lost that detail before encoding. The cost is GPU time on the rendering PC, which normally has it to spare. SteamVR's resolution setting on the rendering PC changes this render size, not the stream.

Bitrate ceiling. Automatic bitrate is at most 0.30 (HEVC), 0.42 (H.264) or 0.25 (AV1) bit per encoded pixel per frame, within 20-300 Mbit/s (`launcher/stream_budget.py`). A link's capacity is a limit, not a target: beyond a few tenths of a bit per pixel the picture barely improves while every frame costs more to send and decode. The governor manages the rate below that ceiling during the session (section 7.3).

7.2 Pipeline policy

The stream is H.264, HEVC or AV1, SDR; HEVC and AV1 are 10-bit when the option is on (P010 surfaces sampled as 16-bit on the GPU). On the AMD path the host converts to Y′CbCr itself (BT.709, full range), first as plain SDR, checks a test picture of known colours at every stream start, and signals what it measured, or streams 8-bit and says why (`VROST_COLOR`, `VROST_COLOR_PROBE`, `VROST_COLOR_FALLBACK`); this keeps black black on OLED panels, which a range mismatch turns grey. The client decodes with the signalled matrix and range, samples chroma at its signalled position and dithers the 8-bit output; the video details show bit depth and range. HDR is outside this contract.

Hardware-decoded frames take one compute pass per shown frame (`fused.comp`). When a frame is decoded its planes are copied into images the client owns and the decoder's surface is released. When it is shown, the pass writes the image SteamVR reads, per output pixel: the re-aim homography, ALVR's foveation inverse with its analytic slope, luma by Catmull-Rom in the sharp region blending to a B-spline in the reduced edges (no fringes where the picture is enlarged), optional contrast-adaptive sharpening of luma only where an output pixel spans a whole encoded texel, chroma at its siting, colour conversion, dither and the fade outside the rendered view. One pass writes one image and resamples once, which is both lighter and sharper than converting and then re-aiming. All constants come from a uniform buffer, so one shader runs on every Vulkan driver, including those with only the minimum 128 bytes of push constants. Sharpening (Off, Low, Medium, High = 0, 35, 70, 120 %; default Medium) applies at once and is separate from foveated encoding. A two-pass fallback (One-pass frame path off) and CPU decoding (diagnosis only; it turns foveation off) remain.

Packet, decode and presentation queues are bounded, and decode and conversion are separated so slow GPU work does not block receiving. Selection favours timely frames with their correct source poses, and image reuse never races the compositor.

A brief interruption reuses the last valid frame: fresh frames are those under about 100 ms old, and for up to one second the last frame is shown re-aimed to the current head rotation, so a frame waiting for its keyframe does not flash grey. The Waiting for video card shows after half a second; longer loss or invalid tracking uses a neutral scene rather than the local Home. SteamVR's own error visuals can still appear on runtime or GPU failures.

H.264 uses CABAC. An encoder priority chooses the host's preset (lowest latency by default). The stream itself never changes size or quality silently mid-session; the headset PC may show each eye smaller under load (section 7.3), and says so.

7.3 Measurement and current issue

Frame age is an internal metric (time since the pose was sampled), not motion-to-photon latency. Each one-second window is classified as `full`, `client_throttled` (the local SteamVR running below refresh), `source_limited` (fewer frames arriving) or `late_or_paired` and logged as `PRESENTATION_CADENCE`. The host keeps a 5-second window of ALVR's per-frame breakdown (render, encode, network, decode, display wait), and the client shows it as a quality line with how long decoded frames wait for the headset's pickup. Once-per-second summaries can hide short stalls and are not per-frame percentiles.

The open issue is the rendering PC's occasional stalls: 40-80 ms without a new frame every half minute or so while a game runs, with the capture timing out on SteamVR's frame. They cost no bitrate (below), but their cause is not yet known.

Foveated encoding (patches 0010, 0012, 0016, 0017, 0019)

Presets set ALVR's per-axis centre size and edge ratio: Subtle (0.70 × 0.65 at 2:1), Balanced (0.55 × 0.50 at 3:1), and Eye-tracked with a Sharp region of Small 0.30, Medium 0.40 or Large 0.55 and an outside of 3:1, 4:1 or 5:1. At 2560 per eye Medium/4:1 encodes about 1408 per eye, some 70 % fewer pixels. Auto chooses Subtle, and Eye-tracked for a Beyond 2e with calibrated gaze. The host compresses before encoding and sends each frame's centres; the client's frame pass undoes it per eye.

- **Where the region sits.** Without fresh gaze (under 150 ms old) the region centres on each eye's optical centre. With gaze it follows the gaze, but moves only when the gaze leaves a comfort zone around the region's middle (`gaze_comfort`, default half the region), then goes straight to it: tracker noise and small eye movements otherwise kept shifting the region's visible edge.
- **Through pauses.** A blink or skipped camera pictures keep the region where it was for `gaze_hold_ms` (default 1 s) before it returns to the optical centre, so the user cannot look straight at the reduced area during a pause.
- **The reduced area filtered.** The host compresses from a mip-mapped, anisotropically sampled copy of the frame, so detail in the edges does not alias and sparkle as the head and gaze move.
- **Diagnostics.** The driver's `VROST_GAZE_FOVEATION` line counts frames on the gaze, held and centred, pauses and how far the gaze reached. The client's live eye-tracked view (page and Game Mode) draws both eyes with the sharp region and gaze point; it is kept in memory only, shared with the host page only during the client's own session, and never logged. When the region does not follow the eyes, the page says why (`eye_state`: eye tracking off, no cameras, not set up or calibrated, starting, cameras, no gaze, CPU decoding).

Network test, pacing and parity (patches 0008, 0009, 0011, 0021)

The network test sends frame-sized UDP bursts from the host to the requesting client only, between sessions, at rising rates (80 to 900 Mbit/s after a warm-up step), and keeps the result per host and adapter. It measures the network rather than the companion: on Linux arrival times come from the kernel and the socket's own drops are counted apart, every datagram carries its send time so the host's own sending time is subtracted, and a step is late when what the network added to a frame's spread reaches 0.6 of a frame interval. A test that finds no clean rate keeps an earlier clean result for up to three tries and is repeated before the next session.

- **Pacing.** The host paces each frame: on Ethernet at 70 % of the test's highest loss-free rate (`PACING_HEADROOM`), so other traffic cannot tip frames into loss, or 60 % of a USB adapter's link speed without a test; on Wi-Fi at the clean test rate, since real Wi-Fi throughput is often under half the radio's rate. On a wired link the automatic bitrate is at most 40 % of the pace, so an average frame arrives within 40 % of a frame interval.
- **Parity, not retransmission.** The host adds one XOR parity piece per group of up to 10 video pieces of a frame, or up to 5 for a wired headset PC, where losses come in clumps; groups are interleaved across the frame so a short run of losses spreads over several groups. The client rebuilds one missing piece per group as soon as the rest arrives, with no round trip. Game audio is sent twice. Parity travels on its own stream ID (the data stream's with the top bit set), so an older client ignores it and an older host sends none; the cost is about a tenth more video data (a fifth when wired). Rebuilt on the way counts repaired frames.
- **Gradual recovery.** Optional encoder intra refresh (NVENC, and AMF via patch 0011) lets the client relax keyframe requests to three consecutive lost frames once the host confirms it is active.

Bitrate governor (client and host companions)

The bitrate is the one lever the client has during a session; size and codec stay fixed until the next Start. `BitrateGovernor` judges each streaming second from the native `TIMING` line, after the first 8 s:

- **Down** 20 % on 2 losses, or 5 troubled seconds in 10 (a loss; decode time over 42 % of the frame interval on average or 65 % at the 95th percentile; or late frames with uneven arrival), never below 35 % of the ceiling or 20 Mbit/s. **Up** 10 % after 40 clean seconds, that patience doubling with each

step down, up to 160 s.

- **Steps that did not help are undone.** A step taken for slow decoding alone, or for uneven arrival alone, is checked afterwards; if decoding is not faster, or arrival not more even, the bitrate goes back up and that cause stops counting for the session. Short Wi-Fi radio pauses and decoders whose time follows picture size, not bitrate, otherwise cost picture for nothing.
- **Frames that never arrived are not the bitrate's.** A second in which frames were missing without network loss (the rendering PC's stalls, a game loading) is neither late nor uneven for the governor, since no lower bitrate brings frames that were never sent.
- **A fresh start after a raised GPU clock.** When the session tuning raises the headset PC's GPU clock, earlier steps down may have been the graphics', so the bitrate returns to the session's start and the patience resets.
- **Where the last session settled.** At Stop the bitrate a session spent most judged seconds at (two minutes at least) is kept per stream (codec, encoded size, refresh, wired or Wi-Fi), with whether its decoding followed the picture size; the next session of that stream starts there, within its floor and its own start, so sessions stop relearning it.
- **Live on the host.** Heartbeats carry the wanted bitrate; the host applies it at most every 2 s through ALVR's `session/values` interface, which ALVR's bitrate manager reads for every frame, with no patch or restart. Against a host without `bitrate-control` the starting bitrate stays and the page says so.

Performance balance and the headset PC's limits

Handheld headset PCs share one power budget between processor and GPU, and their defaults often hold the GPU at a low clock. The companion samples clocks, load and power during sessions and acts in three ways:

- **Session tuning (Bazzite, through steamos-manager).** Only while video streams: the TDP goes to its maximum on the charger; the manual GPU clock is raised in steps when frames run late with the GPU at a low clock and busy, and lowered when the processor is held below 75 % of its own minimum clock or eye tracking loses rate. The clock a session needed is remembered per PC and power source and held from the next session's start. Everything is restored at Stop, or from a receipt after a crash, and a value changed elsewhere is left alone.
- **Advice.** A GPU held back by a power limit or fixed clock, a processor held below its own minimum, or a session short of frames while drawing under three quarters of the last smooth session's power (a lower power limit than before) is named on the page, in Game Mode and in the headset with the measured numbers and the fix (raise the TDP limit on the charger, leave the GPU clock automatic; Set the GPU clock to automatic as one step through steamos-manager). On Windows the advice is judged from what Windows reports and has no one-tap fix.
- **Showing smaller instead of half rate** (`launcher/output_scale.py`). When SteamVR runs the stream below refresh while frames arrive and the GPU is the bottleneck (at least 75 % of the refresh interval), the client shows each eye at 90 %, then 80 %, from the next frame, and returns a step after a minute with room. Steps that fall behind again are backed off progressively, so a PC at its limit does not swing back and forth. The stream is unchanged, so the detail still arrives and is resampled once. Keep the frame rate when this PC falls behind turns it off. A stream larger than SteamVR's recommended size is always shown at that size.

When the limit persists the page offers the headset's smaller size for the next Start, the SteamVR reprojection repair when it is off, or closing other programs.

A lighter frame pass

The frame pass grows with the shown size, so on integrated GPUs it was the part to make lighter for a headset's full panel resolution:

- **Per-frame foveation constants.** The foveation inverse's terms are the same for a whole frame, so `fusedFrameConstants` (`client/device/conversion.hpp`) works them out per frame, eye and axis region in double precision; a pixel then needs one square root and one reciprocal per axis. The compiler's estimate of work per pixel fell by about a third, and each eye is its own layer of workgroups.
- **Only what the lenses show.** The client reads SteamVR's visible-area mesh and its lens distortion outline once at start (`client/device/visible_area.hpp`) and limits each output row to what both show, widened by 3 % because a late frame is shown re-aimed. The mesh's orientation, which SteamVR does not document, is checked against the distortion outline, and a description that would leave less than half an eye is ignored. Skip what the lenses never show turns it off.
- **Measured.** GPU time stamps around the pass and the plane copy give `frame_pass_gpu_*` and `plane_copy_gpu_*` on `TIMING`, shown beside SteamVR's own GPU time.

Pose timing, prediction and re-aim

The host renders each frame with the newest head pose received and extrapolates it by one measured latency, so the client predicts nothing twice: it samples current poses and labels them with the target time (`now` plus ALVR's measured pipeline latency, capped). Poses are sampled on a grid locked to the local display (evenly spaced, at most 3.5 ms apart, re-anchored to vsync once a second, shifted half a step when the host's pick point sits on a sample), because a free-running timer that does not divide the frame period makes head-locked content shake.

- **Controller acceleration** (`client/device/acceleration.hpp`). The host's first-order step lands short or long by half the acceleration times the latency squared for a hand speeding up or slowing down, which is most of a swing. The client estimates each controller's acceleration from consecutive samples (smoothed over 12 ms, bounded, restarted after a gap) and sends $p - a\,h^2/2$ and $v + a\,h$, and the same for orientation, so the host's own step yields a second-order prediction with no change on the wire. `prediction_acceleration` in the settings file (native `--no-acceleration`) turns it off. A change of direction within the latency remains unpredictable on either PC.
- **Gaze prediction** (`client/device/gaze_prediction.hpp`). During a head turn the eyes counter-rotate, so a gaze sample a latency old lands behind where the user looks. The client turns each sample by the head's angular velocity over its age plus the host's step, bounded at 0.35 rad.
- **Prediction check.** For every controller sample the client repeats the host's extrapolation on what it sent and records the distance to where the controller really is (`INPUT` lines, timeline, `controller_prediction` in reports), and logs the distance between two controllers held still, which the host logs as received and as submitted (`VROST_CONTROLLER_PAIR`, patch 0020); equal distances show the poses reach the host's SteamVR as tracked.
- **Re-aim.** Before submission the frame pass can turn each frame from its source pose to the pose the compositor will display it with (rotation only; translation needs depth) and submits it labelled with the display pose. An alignment check at start detects a compositor that already re-aims submitted frames and switches the client's re-aim off. Turning prediction off timestamps poses at sampling, and the host then does not extrapolate them (patch 0014).

Encoder output and frame-timing lock (patches 0013, 0007)

The AMF path forwards every finished frame, oldest first, on each submission and waits within one frame's budget only until a slot is free; a frame in flight for 250 ms resets the encoder. Upstream forwarded at most one output per submission, so one slow frame left the encoder behind for the rest of the session. The client's late-frame catch-up waits at most min(3 ms, a quarter of the refresh interval, time to the compositor's deadline minus 2 ms).

ALVR's virtual vsync free-runs from connection, so its phase against the physical panel is random per session and drifts. Patch 0007 steers it from the client's per-frame decoded to pickup statistic with circular statistics and a bounded PI controller, so frames land a margin (2.5 ms plus twice the measured spread) before pickup; it acquires in one step, then corrects at most 0.5 ms per update. The companion

enables it per session for a driver whose manifest lists the patch. Simulation shows no repeats or drops at any initial phase with ±300 ppm drift; the quality line reports the lock steering in sessions.

8. Tracking, input, menus and play area

Head and controller poses come from the client's runtime and drive the host's virtual devices; timing and source-frame association stay consistent through prediction and presentation (section 7.3). Frame timestamps are the strictly increasing target times, while freshness, hold windows and frame age use local acquisition time. Controller input, skeletons and proximity follow the streaming session, not momentary head-pose validity or compositor focus. Both hands carry buttons, touches, sticks, trigger and grip, native skeletal data (WithController and WithoutController ranges) and haptics; hand identity and validity survive reconnects and transient device loss.

Finger fidelity depends on the Index sensors and the runtime's estimation as well as forwarding, so compare a suspect movement locally with the same game before diagnosing transport.

The menu is the host's SteamVR dashboard. The client suppresses its own dashboard and forwards the System button as one whole press: while it is held the local SteamVR takes input focus (for its own long-press recentre) and every action then reads as released, so the client also reads SteamVR's own button events, which it asks SteamVR to send to every application. A press is forwarded for at least 0.12 s and ends on a release read either way; a loss of focus under 0.5 s keeps every other control at its last value, so a short menu press does not drop what the user holds. A short press opens the host's dashboard and a long one recentres there (confirmed on hardware). Each press is logged (`SYSTEM_BUTTON`). Left B plus thumbstick click and the page's Open host menu are alternatives.

The local SteamVR room setup is authoritative for physical safety and must be completed before room-scale use. The bridge transfers the rectangular play-area size and standing transform to the host and verifies them; it does not clone irregular chaperone polygons or replace the client's boundary system.

9. Audio and optional eye tracking

9.1 Audio

Game audio is captured on the host from Windows' loopback stream directly (patch 0023): silent packets are sent as silence, gaps are counted, the thread runs in Windows' Pro Audio class, a silent render stream keeps the audio engine running through quiet moments, and surround is folded to stereo by speaker position with a soft limit. The device's own volume is applied before that limit, ramped over each packet: the loopback is the mix before the endpoint's volume, and that volume is what SteamVR's dashboard sets for the headset's device; mute is not applied, since the host mutes the device itself while streaming. ALVR's own capture, which hid the packet flags and sent nothing while silent, remains the fallback. Both clients play it through VROSTBridge's own player (patch 0022): the device opened at the stream's rate where possible (otherwise a windowed-sinc converter), the buffer held at its target (30 ms wired, 50 ms Wi-Fi) by a speed correction of at most ±0.5 % so clock drift never empties or overfills it, lost packets cross-faded, and a dry buffer raising its target by 10 ms (up to 150 ms, relaxing after a calm minute) only when the gap was within 30 ms of the target, so a stall of the whole stream doesn't leave the sound late. Clean game audio is confirmed on hardware.

On Bazzite the client makes the headset's speakers and microphone the default devices for the session and restores the previous defaults afterwards; on Windows SteamVR does that itself. Microphone forwarding is opt-in. Windows lists a microphone only when a signed audio driver provides one, so the host page's Headset microphone card offers Set up headset microphone: it downloads VB-CABLE (VB-Audio Software, donationware) from VB-Audio's own server only on request, accepts it only at the pinned SHA-256 or with a valid Authenticode signature by the pinned publisher, holds the files

unchanged until its own setup program has run (opened with a UAC prompt naming VB-Audio; its licence does not allow bundling), then deletes them. A missing microphone warns without stopping video or game audio.

9.2 Eye tracking (Beyond 2e)

Gaze inference is Baballonia (Project Babble, Babble Software Distribution License 1.0, non-commercial), which owns the models and calibration; VROSTBridge accepts its VRChat-style OSC output on 127.0.0.1:9005 and relays a combined gaze direction. Camera images never leave the headset PC. The user guide is `docs/BEYOND_EYE_TRACKING.md`.

Cameras (Bazzite). The bundled go-bsb-cams helper serves the eye cameras as MJPEG on `http://127.0.0.1:8880/stream` through VROSTBridge's own server. It passes the cameras' JPEG pictures on as sent, at the camera's fastest interval, after a structural check (and a full decode of a sample of them), because decoding and re-encoding each picture cost a whole core on a handheld and queued pictures. Damaged pictures are skipped and counted, and the camera is reopened only after a real gap, with `STATS` and `CAMERA` lines in the log. Baballonia's eye filter is set once from its defaults to follow a movement within about two camera pictures. Allow eye camera access writes a udev rule through one pkexec prompt.

Set-up and running. After the user accepts Baballonia's licence, the companion downloads the pinned release (size and SHA-256 checked), unpacks it safely into its own folder with a receipt, and configures Baballonia while it is closed (cameras, eye areas, OSC output, no auto-start or update check, processor for its model). It is kept across VROSTBridge updates and, by default, uninstalls. The session (Start with Eye tracking on) and the user (Open Baballonia) are its owners; one worker reconciles them every 2 s so Start and Stop never wait. Order matters: the relay opens first (Baballonia's OSC sender stops for good after one refused send), the camera helper starts and delivers a picture, then Baballonia starts as its own user unit outside Steam's app scope, at a lower priority so the stream's own work goes first. A watcher ends every helper when the companion goes. A camera helper that ends, or 25 s without a sample, restarts both (never during a calibration).

Calibration over the stream. Baballonia shows its calibration in the headset only while SteamVR runs, so calibration happens during a session. Its OpenXR calibration would be a new SteamVR scene application and SteamVR would quit the stream, so VROSTBridge replaces the file Baballonia starts with a small launcher that runs the same calibration as a SteamVR overlay (`--use-openvr`), as Baballonia does on Windows, while the stream keeps running. SteamVR's client takes an overlay's Vulkan texture through external-memory and semaphore commands that Godot does not enable, so VROSTBridge's own Vulkan layer, `VK_LAYER_VROSTBRIDGE_openvr_textures` (`client/platform/linux/openvr_texture_layer.c`, libc only, MIT), enables the extensions the driver offers and the program did not ask for; `VROSTBRIDGE_VULKAN_LAYER=off` disables it. Stopping the session during training would close Baballonia, so the page says to keep the session running until it has finished.

The session's choice. The sharp region follows the eyes only when, at Start, Eye tracking is on, the headset is a Beyond (or not yet known) and Baballonia is set up and calibrated; otherwise Eye-tracked falls back to Balanced. A calibration saved during a start applies from the next Start, so both PCs agree. Gaze reach reports how far the gaze went each way this session (Baballonia reports at most 45°, and only as far as its calibration went), with advice to calibrate again turning the head further. Gaze centring takes the median of the session's gaze beyond a resting band, caps it, and remembers it per calibration; Use previous calibration returns to the model before. Reports carry the eye-tracking status and Baballonia's log tail, never gaze samples.

Windows. The same switch, licence, owners, relay, recovery and status. Windows lists the eye cameras as an ordinary camera (`BigEye`) that Baballonia reads itself, so there is no camera helper, port 8880 or udev rule; Bigscreen's own eye-tracking app must be closed while Baballonia runs. Baballonia's pinned Windows installer runs privately (section 4.4) and its calibration shows over the stream by itself. Gaze reaches the native client over loopback UDP.

The relay accepts centre or binocular pitch/yaw messages, rejects malformed or non-finite data, and forwards a normalised combined orientation. Per-eye vergence, confidence and eyelid state are not carried; missing gaze is not a blink. The native 50 ms freshness cutoff starts at local receipt. The host uses gaze for foveated encoding and, when the session asks (`session-gaze`), for VRChat's OSC eye tracking; no OpenXR gaze extension or foveated rendering for games is claimed. Ordinary streaming stays usable when gaze is off or unavailable.

10. Network and trust model

Endpoint	Purpose	Scope
UDP 9946	Host discovery and replies	Private LAN interfaces
TCP 9946	Pair/start/heartbeat/stop/status broker	Session API with bearer credentials
TCP 9943 / UDP 9944	Native ALVR connection and stream	Selected host/client path
UDP 5353	Native service discovery	Local network
TCP 9945	Read-only client diagnostics	Recognised peer; bounded grace after disconnect
TCP 9947 (client), 9948 (host); any free port if taken	Local page and authenticated controls	Loopback only
UDP 9 / 7 outbound	Wake-on-LAN magic packets	LAN broadcast and the host's last address
UDP 9944 between sessions	Network test from host to the requesting client	Paired client that asked; never during a session
TCP 8880 / UDP 9005 (client)	Eye cameras' MJPEG stream; Baballonia's OSC to the relay	127.0.0.1 only, while eye tracking runs (no 8880 on Windows)
UDP, a free port per session (Windows client)	Gaze relay to the native client	127.0.0.1 only

Firewall. Host and Windows client rules are created by their own scripts after one administrator prompt, for exactly the ports above and the programs that listen, with `-Profile Any -RemoteAddress LocalSubnet`: they work on any Windows network type but only from the local subnet. An upgrade updates existing rules with one prompt. The Bazzite helper opens its defined inbound set on the observed home subnet. No installer disables a firewall or changes the network category.

Wi-Fi. On Bazzite a Wi-Fi session is held on one access point (the BSSID set on the active connection in memory only, restored at Stop or the next start), because NetworkManager otherwise scans every 300 s with several access points and each scan stalls the picture; the lock is skipped where it cannot help or would break multi-link Wi-Fi. Power saving is turned off for the session where NetworkManager allows, with a one-time permanent option on the page. On Windows the Native Wifi API puts the adapter in media streaming mode with background scanning off. The host can mark video for Wi-Fi priority (DSCP Expedited Forwarding; an optional Windows QoS policy after administrator approval).

One network, two adapters (Bazzite). A headset PC on its home network by cable and Wi-Fi at once has two addresses on one subnet, and Linux by default answers ARP for either address on either adapter, so the rendering PC may send the cable's video through the Wi-Fi card; that stalled the picture whenever Wi-Fi scanned. When this PC is on its network more than once, the page offers Fix this PC's

network settings (`launcher/network_settings.py`): `arp_ignore` 1 and `arp_announce` 2 (each adapter answers only for its own address), plus a larger receive buffer limit only if the stream's socket would get less than 4 MiB (twice `rmem_max`), written to `/etc/sysctl.d/70-vrostbridge-network.conf` through one pkexec prompt and kept across reboots. With it a wired stream leaves Wi-Fi connected (`wifi_others begin(wired, separated)`; always so on Windows). Without it the other Wi-Fi adapter is paused for the session, the host's session port is then tried over the stream adapter, and the pause is undone at once (`resume_now`) if that fails, because some systems drop the cable's route when Wi-Fi goes. Reports carry each adapter's and each socket's drop counters (`adapter_counters`), and a `VIDEO_PATH` note says when the video arrives on another adapter than the one chosen (`adapter_counters.elsewhere()`).

Protocol. The companion protocol is `vrostbridge-session-2`. A host advertises `capabilities` (quality levels, pipeline summary, timing lock) and a `features` list derived from its driver manifest (pacing, gaze foveation, intra refresh, network test, report upload, `bitrate-control`, `session-gaze`, `fovea-view` and others); clients send newer fields only to hosts that advertise them, so mixed versions keep working. The native marker is `vrostbridge-native-1`, with the pinned ALVR wire identity `21-dev14-vrostbridge-native-1`, checked independently of display versions. The broker bounds request sizes (8192 bytes), validates settings and exposes no shell command, URL fetch or path execution. Pairing stores a per-client credential and a host-side hash; status never returns credentials. Report upload is authenticated, size-bounded and rate-limited, and the host stores reports under names it chooses, keeping the 3 newest.

Local page actions are guarded by tokens and Origin/Host checks. Client diagnostics are read-only and peer-restricted. None of this is end-to-end encryption or safety on an untrusted LAN: do not port-forward the service or present it as an Internet service.

11. User interfaces and Game Mode

Each PC has a browser page served on loopback: the host's describes rendering here and a headset elsewhere, the client's the headset here and rendering elsewhere. Both carry the VROSTBridge logo and palette, with the role in the name (VROSTBridge Host, VROSTBridge Client), and the logo, version, licence and independent-project notice in About. Controls put connection state, Start/Stop, readable quality settings, headset detection and Save report first; advanced options stay available without dominating. One page tab is kept: a newer page closes older ones through a `BroadcastChannel`, and stable ports keep them the same origin. The host's Windows tray has a right-click menu, Settings, Exit and status, with an icon that follows the taskbar's theme; a second launch reopens the existing page. The Windows client's tray is described in section 4.4.

In the headset, SteamVR-style status cards and the quality pill are head-locked overlays drawn at up to twice the pixels per degree (within SteamVR's 2 MiB per image) so the lenses shrink them rather than enlarge them.

Game Mode (Bazzite). Gamescope's full-screen Steam session gets a native SDL interface, not a browser or Proton, sharing preferences and session logic with the page, drawn at the display's resolution with the pages' fonts. It has every client setting on scrolling pages (Stream & headset, Motion & timing, Network & host, This PC, Eye tracking). The D-pad, left stick, keyboard, mouse and touch work; on settings lists left and right change a value, held directions repeat, one press moves once, and B goes up a level but never exits. Add to Steam (Desktop Mode) adds a non-Steam shortcut with artwork to the most recent account (byte-exact round trip, backup, receipt) and sets up the integration below; Remove from Steam and uninstall reverse it.

The headset display in Game Mode. A per-user gamescope wrapper keeps a connected headset from becoming the Game Mode screen (headset outputs, recognised by EDID, are ranked after every other output) and reserves the headset's connector with `--lease-connector`. Only head-mounted displays are reserved: known Index and Beyond product codes, and any EDID carrying the kernel's non-desktop markers (Microsoft HMD block or DisplayID head-mounted use), with the kernel's `non-desktop` property as

a fallback. At Start, a lease helper validates the connector and offers SteamVR a private Wayland DRM-lease socket under a plain name (`vrostbridge-headset-<pid>`), and SteamVR starts without gamescope's own display marker so the adapter's is the only Wayland display it sees. If the compositor reports it cannot lease the display, Start stops at once with the reason. Every step (programs connecting, the lease granted, the display SteamVR's server got, the compositor's lines) is logged and kept in `game-mode-last-start.json` for reports. Any wrapper failure runs plain gamescope with the original arguments; the helper never outlives the companion. Physical Game Mode acceptance is the main open Bazzite gate.

12. Installation, upgrades and stored state

The Windows host is a per-user NSIS installer plus a portable package, bundling its launcher runtime, patched driver and notices. The Bazzite client is an extracted ZIP with a desktop installer that installs under `~/.local/share/vrostbridge` without touching the immutable OS image, layering RPMs, Proton or a compiler; it needs x86-64 Linux with glibc 2.39 or later. The Windows client is a per-user NSIS installer (`VROSTBridge-Client-Setup-<version>.exe`) and a portable ZIP, installing to `%LOCALAPPDATA%\VROSTBridge\Client\<version>` without administrator rights on Windows 10 or later x64; it can sit beside the host on one PC with its own folder, uninstall key, Run value and shortcuts. No package is code-signed, so SmartScreen asks once on Windows.

Upgrades verify the staged payload against its checksums (the launcher's `--self-test`), close the old companion through its authenticated local path, copy the verified files (a same-version repair is put back if the copy fails) and only then remove older versions. Repair reinstalls damaged owned files. Only files identifiable as VROSTBridge's own are removed; preferences, pairing, client identity, reports and retained host configuration are preserved. An unknown modified installation produces an actionable failure rather than deleting anything.

State lives in `%LOCALAPPDATA%\VROSTBridge` on Windows and `~/.local/state/VROSTBridge` on Bazzite. Besides preferences and pairing it includes receipts for anything changed for a session (SteamVR settings, power, Wi-Fi, session tuning), `gpu-clock.json`, `base-stations.json`, `gaze-centre.json`, `game-mode-last-start.json` and, on the host, `host-last-session.json`. Uninstall removes owned program files, shortcuts, integrations and firewall rules (one prompt) and asks whether to keep the downloaded Baballonia (kept by default); Baballonia's own settings and calibration, the root-owned camera rule, SteamVR, other ALVR installations and user-edited files stay. Full state removal is a separate deliberate action.

13. Diagnostics and support

Both ends keep bounded logs, start-up milestones, negotiated settings and incidents. The host collects client diagnostics while connected, and the client can send its report to the host, so one place holds both. Every report starts with `report_info` (which end, version, time) and its file name says the same. A host report made with no session since its companion started carries the last session's statistics as `host_last_session`, from `host-last-session.json`, written every minute and at Stop.

Reports establish versions and protocol, headset identity, requested and actual stream format, effective refresh, decode and presentation state, controller validity, network path and adapter counters, start/stop history and errors. Host reports include the driver's own log (filtered of per-frame lines) and its `VROST_*` lines; client reports include `STARTUP` milestones, the camera helper's statistics, head poses sent and missing, the network test's steps, and a `timeline` of the session (one row per 10 s, at most 360: frame rate, losses and repairs, keyframes, presentation incidents, frame age, eye-tracking rate, clocks, load, power, cadence and the processor time of the session's own programs, others summed without names). A report is diagnostic evidence, not a benchmark certificate.

Live eye and world pose streams are never dumped and credentials are masked, but reports can still contain host names, IP addresses, device identities, paths and hardware details, so review them before posting publicly. Include what the user saw, a local-versus-bridge comparison, test duration and whether the runtime restarted.

14. Build, dependencies and source handoff

The source archive holds VROSTBridge's own code, tests, assets, scripts, the ALVR patches, notices and selected native Linux payloads. It is not an offline mirror of every dependency: build scripts fetch pinned upstream sources, and clean native builds need Internet access and platform toolchains. Each release also offers the source split by part (rendering PC, Windows client, Bazzite client), each with a `SOURCE.md`, from the same file list as the whole source (`scripts/package_component_sources.py`).

Component	Pinned baseline / role
ALVR	`f5023f8e78fa14ea57fb1038548d7820289a8524`; host and client-core transport
OpenVR	`0924064316de3effbcd1acf1e309182a2deb1c05`; headers and loaders
Rust	1.97.1; native adapter and host build
Linux FFmpeg	n6.1.4 / `34277e12e80031c7f89494ba543684bc1dd0be8f`; minimal shared decode build (H.264, HEVC, AV1 with VA-API)
Windows FFmpeg	The same n6.1.4 source (SHA-256 checked); minimal shared decode build with D3D11VA, cross-compiled with mingw-w64 GCC 13, libgcc static; arguments in `third_party/ffmpeg/BUILD-Windows.txt`
Windows client toolchain	LLVM 19+ (clang-cl, lld-link) with a licensed Windows SDK and CRT in xwin layout, CMake 3.24+, Ninja, Rust 1.97.1 for `x86_64-pc-windows-msvc`, NSIS 3.09, the host payload's CPython 3.13.12, Wine 9.0 with Mesa's lavapipe for tests
go-bsb-cams	`d0df45de2d9ce1a7f7febf6a2c1313812e5df287`; camera helper
Baballonia	Not shipped; downloaded on request: Linux `Baballonia.x64.v1.1.1.0rc6.tar.xz` (517,439,412 bytes, SHA-256 `06c9ce7ecca19a0fee3ead40ef0c705ca173916e11a3315b99df63d348ea906a`), Windows `Baballonia.Setup.v1.1.1.0rc6.exe` (423,324,292 bytes, SHA-256 `dec2e8fd1e5c844662f99b47680d772a393f6eb2fdf4e4dd3c62853928b2c470`, three installed files also checked)

The ALVR patches, applied in order by `scripts/fetch_upstream.py` after checking revision identity:

Patch	Purpose
0001-0006	Microphone consent, network resilience, native controller fidelity, host capture/AMF, native protocol and build, idempotent chaperone
0007	Frame-timing lock
0008-0012	Client stream statistics and loss policy, pacing and stream options, gaze and optical-centre foveation, AMF intra refresh
0013-0015	AMF output drain, prediction switch, startup display configuration

Patch	Purpose
0016-0019	Foveation prefilter, gaze hold, exact colour and frames without a head pose, gaze comfort zone
0020-host-colour-variants-and-gaze-reach.patch	Plain-SDR colour variant, gaze reach and still-controller diagnostics
0021-0023	Parity for video and audio, the clients' audio player, the host's game-audio capture

Do not substitute current upstream builds or an unrelated ALVR dashboard; native protocol and build-manifest checks reject incompatible payloads.

The host driver builds on Windows x64 with Visual Studio 2022 C++ tools, Git, Python, CMake and the pinned Rust, or by the Linux cross-build `scripts/Build-Host-Cross.py` (Clang/LLD, MSVC ABI). Release drivers are built without ALVR's `gpl` feature. `scripts/host_module_tests.py` builds and tests the driver's VROSTBridge Rust modules on Linux with stand-ins for ALVR's types. Linux builds use C++17, CMake, Rust, FFmpeg, Vulkan, VA-API, libdrm, Wayland, SDL2 and audio libraries. The Windows client is cross-built on Linux in four steps (FFmpeg DLLs, the transport DLL, the native client, the package), and the package builder refuses wrong versions, non-x64 files, dynamic C runtimes, FFmpeg not built from the shipped source and an embedded Python that differs from the host's. See `BUILDING.md` and `packaging/README.md`. Pinned inputs and hash checks aid repeatability; byte-for-byte reproducibility across arbitrary systems is not claimed.

15. Attribution and publication readiness

Keep `THIRD_PARTY.md`, `OPEN_SOURCE_CREDITS.md`, licence files, vendored notices and dependency source and build records. ALVR is reused under MIT, OpenVR under BSD-3-Clause, the minimal FFmpeg builds (Linux and Windows) under LGPL-2.1-or-later, each package carrying FFmpeg's unmodified source and build notes, and the camera helper under GPL-2.0 with its changes listed beside its source. The Windows packages carry the mingw-w64 runtime notice; libgcc is under the GCC Runtime Library Exception. The host driver is built without FFmpeg. A credits page alone does not meet every obligation.

VROSTBridge's own code is MIT-licensed (`LICENSE`, copyright JagerBombastic). `NOTICE` carries the key attributions, including NVIDIA's required statement for the Video Codec SDK sample code in the driver; `THIRD_PARTY.md` lists every shipped component with its licence. Baballonia and VB-CABLE stay downloads from their authors at the user's request, under their own licences. Test data uses made-up LAN addresses and placeholder serials. `docs/PUBLISHING.md` is the checklist for a public repository.

This is an engineering review, not legal advice. VROSTBridge is an independent open-source project, not affiliated with, sponsored by or endorsed by Valve Corporation, Bigscreen, any other headset, tracking or platform maker, the ALVR project or Project Babble. Product names and trademarks belong to their owners and identify compatibility only. The logo and icons are the project's own artwork (`brand/`), licensed CC BY-ND 4.0 (`brand/LICENSE.md`: share unchanged with credit; forks use their own name and logo); no trademark registration is claimed.

16. Validation and release acceptance

16.1 Automated gates

Native contract tests cover frame selection, geometry and IPD, projection round trips, skeletal and input handling (including the menu button), readiness, chaperone, the DRM-lease protocol, the frame pass against its CPU model (`tests/fused_shader_tests.cpp`), the visible area, the shown size, the client's whole frame path with a stand-in SteamVR (`tests/video_pass_tests.cpp`), controller acceleration and gaze prediction. Python tests cover profile validation, lifecycle ownership, network and pairing policy, the network test, pacing and governor (replaying recorded sessions), the performance balance and base stations against stand-in system services, eye tracking with stand-ins, reports, install maintenance and the UI settings; the page has its own JavaScript checks. Each release adds a `tests/test_vrostbridge_<version>.py` for what it changed. The driver's Rust modules and colour rules are tested on Linux, the patched ALVR crates' own tests cover packet repair and the audio player, and the camera helper's Go tests run under the race detector. Packaging checks cover executable start-up, frozen resources, authenticated local HTTP, staged manifest integrity and archive CRCs; upgrade and repair tests prove user state survives.

The Windows client's gates all run on Linux: its Python modules with stand-ins for Windows calls, the C++ contract tests cross-built and run under Wine with Mesa's lavapipe, the package builder's checks and Wine runs of the installer, uninstaller and companion. Wine has no D3D11 video decoder, so D3D11VA and the real Firewall, Wi-Fi, power and Baballonia installer calls are hardware gates.

Documentation checks verify internal links and the rendered PDF; they never reclassify earlier hardware tests as newly run.

16.2 Physical acceptance matrix

Rows are grouped by area; Status says whether the behaviour is already confirmed on hardware.

Area	Required observation	Status
Index session, Ethernet and Wi-Fi	Correct stereo, stable head motion, actual refresh and stream size shown, audio intact	Confirmed
Index display modes	120 and 144 Hz run as chosen; SteamVR's own setting is back after Stop	Pending
Beyond session	Streams the mode it is in (2560 at 75 Hz, 1920 at 90 Hz); no headset-ready timeout	Confirmed at 75 Hz; 90 Hz pending
Beyond projection	Optical centre and FOV correct; stable scale and alignment	Pending
Picture at the same stream	Stream: line names HEVC on an Index, a render size above the stream size and the higher ceiling; far field sharper	Pending
Exact colour	With 10-bit on, black as black as on the PC; `VR0ST_C0L0R … verified=1` or a stated 8-bit reason	Pending
Steady picture	Head-locked text and held objects still while turning; timing lock steering; no repeat/drop judder across Stop/Start	Confirmed (motion); lock pending
Native on the headset PC	Native with eye-tracked foveation holds refresh; frame pass and plane copy timed; nothing at the lens edges cut off	Confirmed

Area	Required observation	Status
Keeping the frame rate	Where the PC cannot hold refresh, Shown at 90 % (or 80 %) instead of half rate, full size back when there is room	Pending
Graphics and power limits	Warnings name a held-back GPU or processor with measured numbers; session tuning raises the clock and restores the user's settings after Stop and after a crash; the clock is held from the next start	Partly confirmed
Network test, pacing and bitrate	Plausible test on Ethernet and Wi-Fi; few lost frames; any `BITRATE` step has a reason and the host follows it; steps that did not help are undone	Confirmed
Packet repair	Rebuilt on the way counts while lost frames stay rare, on Wi-Fi and wired	Confirmed
One network, two adapters (Bazzite)	After Fix this PC's network settings, Wi-Fi stays connected, no `VIDEO_PATH` note, the video on the stream adapter, no periodic stall	Pending
Any Windows network type	With the rendering PC's network Public, a paired client starts; a new one is refused pairing with the reason	Pending
Wi-Fi without stalls	No periodic stall over 15+ minutes on Wi-Fi; Wi-Fi normal after Stop	Pending
Controllers	30+ minute soak: both hands keep actions and skeletons; held inputs survive brief occlusion; sleep/reconnect recovers; haptics	Partly confirmed
Hands where they were swung	In a rhythm game notes swung at register; the prediction check's 95th percentile well below earlier sessions	Pending
Menu button	Short press opens the host's dashboard every time; long press recentres; held trigger or grip stays held	Confirmed (short press)
Game audio and microphone	Minutes of music or dialogue without crackle; host microphone set up with VB-CABLE and working in a voice app	Audio confirmed; microphone pending
Lifecycle	Repeated cold/warm Start/Stop clean on both PCs; a stop during SteamVR's start ends within about ten seconds; compositor crash restarts once	Partly confirmed
Power and wake	Host stays awake in sessions, wakes by Wake-on-LAN, sleeps after the chosen time; base stations on at Start and off at closing	Pending
Host last session	A host report made after the host's companion restarted carries `host_last_session`	Pending
IPD and room setup	IPD changes without discontinuity; host standing origin and rectangle agree with the local boundary	Pending
Loss and recovery	Brief loss holds the view re-aimed; longer loss is neutral; the stream returns after a network drop	Partly confirmed

Area	Required observation	Status
Eye tracking (Bazzite, Desktop Mode)	Set-up verifies Baballonia; samples near the camera rate; calibration shown over the stream with the session kept; region follows the eyes, holds through blinks and stays put for small movements; Gaze reach plausible	Confirmed (calibration, following); rate and reach pending
Gaze during head turns	The sharp region stays on what is looked at while the head turns	Pending
Game Mode	Built-in screen stays on; headset reserved and leased to SteamVR; session streams; every setting reachable; B never exits; return to desktop	Interface confirmed; session pending
Windows client	Installs without administrator rights; firewall prompt; network test and a session of several minutes with D3D11VA and a small copy time; tray and sign-in; power and Wi-Fi back after Stop; eye tracking with BigEye and calibration over the stream	Pending
Install, repair, uninstall	Fresh and upgrade paths keep preferences and touch no unrelated software	Confirmed

For performance comparisons use the same scene, resolution, codec, bitrate and refresh, and record both PCs' hardware, route, versions and reports. Compare unique-frame cadence and percentiles only where the sampling supports them; there is no universal latency promise.

17. Priorities and future work

1. Confirm 1.0.0.15 on hardware: the dashboard volume in the headset, the safe-mode message and its fix, the USB and controller notes, the bitrate starting where it settled and the audio's aim through a long session; and what 1.0.0.14 left open: the far field at the same stream, a Public network on the rendering PC, the sharp region during head turns, and `host_last_session`.
2. Find the cause of the rendering PC's stalls: the host report's `capture_stalls` shows whether its capture timeouts follow the encoder's network send, as the logged ones did; compare sessions with eye tracking off and with another game.
3. Close the open hardware gates of earlier releases: an Index at 120 and 144 Hz, a stop during SteamVR's start, Keep the frame rate under load, and the controller soak and lifecycle repeats.
4. Get a Game Mode session streaming, using the kept start report to see where the display handover fails.
5. Run the Windows client on Windows hardware: install, firewall, network test, a session with D3D11VA, tray, power and Wi-Fi restore, and eye tracking.
6. Confirm the Beyond projection and Index regression, then decide whether the evidence supports a stable release label.
7. Later, if reports show they matter: decoding without the plane copy (shared D3D11 texture or Vulkan Video), the lens area on further headsets, further pacing ideas measured against a baseline.

Code-signed installers, Internet transport, HDR and dynamic foveated rendering for games remain separately scoped. Keep working Index behaviour as the regression baseline while extending Beyond support.